

Document number:	P0541R1
Date:	2017-07-10
Project:	C++ Extensions for Ranges
Reply-to:	Eric Niebler < eniebler@boost.org >
Audience:	Library Working Group

Ranges TS: Post-Increment on Input and Output Iterators

- [1 Synopsis](#)
- [2 Revision History](#)
 - [2.1 R1](#)
- [3 Input Iterators](#)
 - [3.1 The Problem](#)
 - [3.1.1 Operator Expense](#)
 - [3.1.2 Adaptor Complications](#)
 - [3.2 Alternative Solutions](#)
 - [3.2.1 Do nothing](#)
 - [3.2.2 Remove postfix increment entirely](#)
- [4 Output Iterators](#)
- [5 Proposed Design](#)
- [6 Acknowledgements](#)

1 Synopsis

The requirement for `InputIterators` to provide a post-increment operator that returns a type satisfying the `Readable` concept presents significant implementation challenges for many Input iterators, and makes some impossible to implement. This paper suggests removing the requirement for `InputIterators`. Specifically, we propose lifting the requirement that the return type of the post-increment operator for `InputIterator` satisfy `Readable`. We retain the requirement for forward iterators.

Note that nothing would prevent a given input iterator from supporting the syntax `*i++` or prevent non-generic code from making use of that syntax. The only change would be that generic code that operates on input iterators would need to change statements like `fn(*i++)` to `fn(*i); ++i`. This is likely to be more efficient, as is discussed below.

In Issaquah, the problem was discussed and a straw poll taken. The wording of the poll was as follows:

Do we want to loosen the requirement on `InputIterator` post-increment to permit void return?

S	F	F	N	A	S	A
13	6	1	0	1		

2 Revision History

2.1 R1

- Incorporate [the PR for Ranges TS issue #361 “missing semantics for OutputIterator’s writable post-increment result”](#) (per LWG Toronto directive)
- Incorporate [the PR for Ranges TS issue #386 “basic exception guarantee in counted_iterator’s postincrement”](#) (per LWG Toronto directive)

3 Input Iterators

3.1 The Problem

Iterator post-increment requires that the expression `*i++` advances the iterator but returns the previous value. For forward iterators, that is trivially implementable by having the expression `i++` return a copy of the iterator before the application of *pre-increment*; e.g., `{ auto tmp = i; ++i; return tmp; }`. This doesn’t work for `InputIterators` because pre-increment invalidates all copies of the iterator. When considering an iterator like `istreambuf_iterator` it is easy to see why: incrementing reads from a stream, thereby consuming the range as it is traversed.

In order for an iterator like `istreambuf_iterator` to give the expression `*i++` the required semantics, `i++` must return a proxy object that caches the previous value within it, and returns a reference to the previous value from an overloaded unary operator`*`. This proxy object must satisfy the `Readable` concept for `istreambuf_iterator` to satisfy the `InputIterator` concept in the Ranges TS.

3.1.1 Operator Expense

`Readable` requires `Movable` and `DefaultConstructible`. For value types that are not themselves `Movable` or `DefaultConstructible`, various hacks are necessary for the proxy object to satisfy the concepts; e.g., storing the previous value in a `std::optional` or on the heap. Types that are not efficiently movable (e.g., `std::array`) are copied into the proxy object at great runtime cost, a cost that is hidden in the innocuous-looking expression `i++`.

Types that are *not* movable simply cannot be cached. An input iterator over non-movable types is simply disallowed by the requirements on `operator++(int)`.

3.1.2 Adaptor Complications

The standard specifies the existence of several iterator adaptors (e.g., `move_iterator`), and future iterations of the Ranges TS will propose many more. `move_iterator` is required to work with input iterators. Consider the difficulty of implementing `move_iterator`'s postfix increment operator. Below is the implementation as specified in the current draft standard.

```
template <InputIterator I>
move_iterator<I> move_iterator<I>::operator++(int) {
    auto tmp = *this;
    ++this->current; // A data member of type I
    return tmp;
}
```

This implementation is obviously wrong for input iterators – it is guaranteed to return an invalid iterator – and yet this is precisely what the standard specifies today. Correcting the problem is non-trivial. `move_iterator`'s postfix increment operator would need to return a proxy for input iterators only. Below is a rough demonstration, expressed in C++17.

```
template <InputIterator I>
auto move_iterator<I>::operator++(int) {
    using _Cat = iterator_category;
    if constexpr (is_base_of_v<forward_iterator_tag, _Cat>) {
        // Forward iterators permit a simple implementation
        return move_iterator{this->current++};
    } else {
        // Input iterators must return a proxy
        using _R = decay_t<decltype(this->current++)>;
        struct __proxy {
            using value_type = value_type_t<_R>;
            using _Ref = reference_t<_R>;
            using reference = conditional_t<
                is_reference_v<_Ref>,
                remove_reference_t<_Ref>&&,
                decay_t<_Ref>>>;
            _R __cache;
            reference operator*() const {
                return static_cast<reference>(*__cache);
            }
        };
        return __proxy{this->current++};
    }
}
```

The very fact that the C++ standard gets this wrong for `move_iterator` demonstrates the complex subtlety that the requirement introduces into the standard iterator concepts.

The problem is even worse for other iterator adaptors. Imagine an adaptor that transforms its base iterator's reference somehow. Just because the iterator knows how to transform the base iterator's reference type is no guarantee that it knows how to transform its postfix-increment proxy reference type. In other words, for some transformation `TFX`, the well-formedness of `TFX(*it)` doesn't ensure the well-formedness of `TFX(*it++)`. That's because the types of the expressions `*it` and `*it++` are not required to be the same for input iterators. For example, `*it` might be an `int&`, but `*it++` might be an `int`.

In other words, due to the inherent oddness of the postfix increment operator, it is impossible to implement a generic iterator adaptor for all valid input iterators.

3.2 Alternative Solutions

The following solutions have been considered and dismissed.

3.2.1 Do nothing

One obvious solution is to simply do nothing. We have lived with the oddness of input iterator’s postfix increment operator long enough. We can continue to live with it in the Ranges TS and beyond. Some generic iterator adaptors will be impossible to get completely correct, but its likely to be corner cases that will fail. Again, probably not a *huge* deal. Despite the preceeding, we have decided to push forward our suggested resolution because with Concepts we have a chance to correct various shortcomings of the iterator concepts. Now is the time for any breaking changes. This change sweeps away the necessity for a lot of needless complexity in iterator implementations, and steers users away from using a surprisingly expensive iterator operation.

With the Ranges TS, we have a chance to try out this change without fully committing to it. Should the lack of `*i++` prove to be an adoption challenge, we would still have time to add it back in should the Ranges TS ever be merged into the IS.

3.2.2 Remove postfix increment entirely

During the Issaquah 2016 meeting when this issue was discussed in LEWG, several in the room expressed support for the notion of simply removing postfix increment for input iterators entirely. (This is in contrast with requiring postfix increment but permitting it to return, say, `void`.) The following straw poll was taken on that question:

Do we want to remove the requirement for `InputIterator` post-increment entirely (for the Ranges TS)?

S	F	F	N	A	S	A
7	6	7	2	0		

Although there does appear to be weak consensus to remove `i++` entirely from the `InputIterator` concept, the consensus to keep `i++` but permit it to return `void` is stronger.

The authors of the Ranges TS strongly prefer keeping the postfix increment operator for input iterators. Consider the following fairly common (anti-)pattern:

```
for (auto i = v.begin(); i != v.end(); i++) {  
    // ...  
}
```

Aside from concerns over the cost of post-increment, there is nothing wrong with this code *per se*, and the authors see no compelling reason to break it. And by permitting `i++` to return `void`, we would actually be mitigating the performance problem.

4 Output Iterators

After considering the issue for input iterators, its natural to wonder what the situation is for output iterators. Should `OutputIterator` require writabilty for the expression `o++`? The present draft of the Ranges TS places no requirements on the expression `o++`, making the expression `*o++ = t` non-portable in generic code. Perhaps we should add it.

On the one hand, output iterators need not cache any previous value in the result of `o++` to give `*o++ = t` the correct semantics. As a result, there is no performance problem with postfix increment, and no known difficulty adapting output iterators. *Not* supporting writability for `o++` would seem to be a needless incompatibiliity with the iterators in the IS. We don’t know of an output iterator for which the postfix increment operator is not completely trivial and efficient.

On the other hand, its hard to say with certainty that there will *never* be an output iterator with an expensive and complicated postfix increment. And supporting `*o++` for output iterators but not for input iterators is inconsistent and confusing.

Avoiding needless breakage *seems* like the more convincing argument. Neither of the authors of the Ranges TS has any strong opinions on the matter, but this paper suggests adding back the writability requirement on `o++`.

5 Proposed Design

Change the definition of `InputIterator` ([`iterators.input`]) as follows:

```
template <class I>
```

```

concept bool InputIterator() {
    return Iterator<I>() &&
        Readable<I>() &&
requires(I i, const I ci){
    requires { typename iterator_category_t<I>; } &&
requires DerivedFrom<iterator_category_t<I>, input_iterator_tag>();
{ i++ }->Readable; // not required to be equality preserving
requires Same<value_type_t<I>, value_type_t<decltype(i++)>>();
{ *ci }->const value_type_t<I>&;
};
}

```

[Editor's note: Note: the line `{ *ci } -> const value_type_t<I>&;` is deleted here as a drive-by fix of [stl2 issue 307](#) in which the edits necessary to support proxy iterators (P0022) were applied incompletely.]

Change the definition of OutputIterator ([iterators.output]) as follows:

```

template <class I, class T>
concept bool OutputIterator() {
    return Iterator<I>() && Writable<I, T>(); &&
requires(I i, T&& t) {
    *i++ = std::forward(t); // not required to be equality preserving
};
}

```

Add a new paragraph between [iterators.output]/1 and paragraph 2:

-?- Let E be an expression such that `decltype(E)` is T , and let i be a dereferenceable object of type I . Then `OutputIterator<I, T>()` is satisfied only if `*i++ = E`; has effects equivalent to:

```

*i = E;
++i;

```

Change the class synopsis of insert_iterator ([insert.iterator]) as follows:

```

namespace std { namespace experimental { namespace ranges { inline namespace v1 {
    template <class Container>
    class insert_iterator {
    public:
        // ... as before
        insert_iterator& operator++(int);
        // ... as before
    };
}
}
}

```

Change [insert.iter.op++] as follows:

```

insert_iterator& operator++();
insert_iterator& operator++(int);

```

1 Returns: `*this`.

[Editor's note: Thus restoring the signature of `insert_iterator`'s postfix increment operator to the version in the IS.]

Change the class synopsis of move_iterator ([move.iterator]) as follows:

```

namespace std { namespace experimental { namespace ranges { inline namespace v1 {
    template <InputIterator I>
    class move_iterator {
    public:
        // ... as before
move_iterator operator++(int);
void operator++(int);
move_iterator operator++(int)
requires ForwardIterator<I>();
        // ... as before
    };
}
}

```

Change [move.iter.op.incr] as follows:

```

move_iterator& operator++();

```

1 Effects: Equivalent to `++current`.

2 Returns: `*this`.

```
void operator++(int);
```

3 Effects: Equivalent to ++current.

```
move_iterator operator++(int)↗  
requires ForwardIterator<I>();
```

4 Effects: Equivalent to:

```
move_iterator tmp = *this;  
++current;  
return tmp;
```

Change the class synopsis of `common_iterator` ([`common.iterator`]) as follows:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {  
    template <Iterator I, Sentinel<I> S>  
        requires !Same<I, S>()  
    class common_iterator {  
    public:  
        // ... as before  
        common_iterator operator++(int);  
        decltype(auto) operator++(int);  
        common_iterator operator++(int)  
        requires ForwardIterator<I>();  
        // ... as before  
    };  
}
```

Change [`common.iter.op.incr`] as follows:

```
common_iterator& operator++();
```

- 1 Requires: `!is_sentinel`.
- 2 Effects: Equivalent to `++iter`.
- 3 Returns: `*this`.

```
decltype(auto) operator++(int);
```

- 4 Requires: `!is_sentinel`.
- 5 Effects: Equivalent to: `return iter++;`

```
common_iterator operator++(int)↗  
requires ForwardIterator<I>();
```

- 6 Requires: `!is_sentinel`.
- 7 Effects: Equivalent to:

```
common_iterator tmp = *this;  
++iter;  
return tmp;
```

[*Editor's note:* For input and output iterators, we return the result of `iter++` directly. That permits `common_iterator`'s postfix increment operator to work correctly with input and output iterators that return proxies (e.g., `istreambuf_iterator`) or references to `*this` (e.g., `insert_iterator`) from their postfix increment operator.]

Change the class synopsis of `counted_iterator` ([`counted.iterator`]) as follows:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {  
    template <Iterator I>  
    class counted_iterator {  
    public:  
        // ... as before  
        counted_iterator operator++(int);  
        decltype(auto) operator++(int);  
        counted_iterator operator++(int)  
        requires ForwardIterator<I>();  
        // ... as before  
    };  
}
```

Change [`counted.iter.op.incr`] as follows:

```
counted_iterator& operator++();
```

- 1 Requires: `cnt > 0`.

2 Effects: Equivalent to:

```
++current;  
--cnt;
```

3 Returns: `*this`.

```
decltype(auto) operator++(int);
```

4 Requires: `cnt > 0`.

5 Effects: Equivalent to:

```
--cnt;  
try { return current++; }  
catch(...) { ++cnt; throw; }
```

```
counted_iterator operator++(int);  
requires ForwardIterator<I>();
```

~~6 Requires: `cnt > 0`.~~

~~7 Effects: Equivalent to:~~

```
counted_iterator tmp = *this;  
<ins>++*this;</ins><del>++current;</del>  
<del>--cnt;</del>  
return tmp;
```

[Editor's note: For input and output iterators, we return the result of `current++` directly. That permits `counted_iterator`'s postfix increment operator to work correctly with input and output iterators that return proxies (e.g., `istreambuf_iterator`) or references to `*this` (e.g., `insert_iterator`) from their postfix increment operator.]

No changes to `istream_iterator` or `istreambuf_iterator`.

[Editor's note: We suggest leaving the postfix increment operators on the `istream(buf)` iterators intact to ease migration to the Ranges TS.]

Change the class synopsis of `ostream_iterator` ([`ostream.iterator`]) as follows:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {  
    template <class T, class charT = char, class traits = char_traits<charT>>  
    class ostream_iterator {  
    public:  
        // ... as before  
        ostream_iterator& operator++(int);  
        // ... as before  
    };  
}
```

Change [`ostream.iter.ops`] as follows:

```
// ... as before  
ostream_iterator& operator++();  
ostream_iterator& operator++(int);
```

3 Returns: `*this`.

[Editor's note: Thus restoring the signature of `ostream_iterator`'s postfix increment operator to the version in the IS.]

Change the class synopsis of `ostreambuf_iterator` ([`ostreambuf.iterator`]) as follows:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {  
    template <class charT, class traits = char_traits<charT>>  
    class ostreambuf_iterator {  
    public:  
        // ... as before  
        ostreambuf_iterator& operator++(int);  
        // ... as before  
    };  
}
```

Change [`ostreambuf.iter.ops`] as follows:

```
// ... as before  
ostreambuf_iterator& operator++();  
ostreambuf_iterator& operator++(int);
```

5 Returns: *this.

[Editor's note: Thus restoring the signature of ostreambuf_iterator's postfix increment operator to the version in the IS.]

6 Acknowledgements

I would like to thank Casey Carter for his review feedback.